Assignment 5 - Recurrences and Hash Tables

#1: Recurrence

$$T(n) = 5T(n/7) + 3n^2$$

Solution

$$T(n) = 5T(n/7) + 3n^2$$
$$= 5[5T(n/7^2) + 3(n/7)^2] + 3n^2$$
$$= 5^2 T(n/7^2) + 15(n/7)^2 + 3n^2$$
$$= 5^2 T(n/7^2) + 3 \cdot 5(n^2/7^2) + 3n^2$$

$$= 5^2[5T(n/7^3) + 3(n/7^2)^2] + 15(n/7)^2 + 3n^2$$
$$= 5^3 T(n/7^3) + 75(n/7^2)^2 + 15(n/7)^2 + 3n^2$$
$$= 5^3 T(n/7^3) + 3 \cdot 5^3(n^2/7^4) + 3 \cdot 5(n^2/7^2) + 3n^2$$

$$= 5^3[5T(n/7^4) + 3(n/7^3)^2] + 75(n/7^2)^2 + 15(n/7)^2 + 3n^2$$
$$= 5^4 T(n/7^4) + 375(n/7^3)^2 + 75(n/7^2)^2 + 15(n/7)^2 + 3n^2$$
$$= 5^4 T(n/7^4) + 3 \cdot 5^3(n^2/7^6) + 3 \cdot 5^2(n^2/7^4) + 3 \cdot 5(n^2/7^2) + 3n^2$$

$$\hookrightarrow 3n^2\left(\frac{5^3}{7^6}\right) \quad \hookrightarrow 3n^2\left(\frac{5^2}{7^4}\right) \quad \hookrightarrow 3n^2(5/7^2)$$

* each term is $3n^2\left(\frac{5}{49}\right)^k$

$$T(n) = 5^k T(n/7^k) + 3n^2 \sum_{i=0}^{k-1} \left(\frac{5}{49}\right)^i$$

Scratch

$$T(n/7) = 5T\left(\frac{n}{7^2}\right) + 3\left(\frac{n}{7}\right)^2$$

$$T(n/7^2) = 5T(n/7^3) + 3(n/7^2)^2$$

$$T(n/7^3) = 5T(n/7^4) + 3(n/7^3)^2$$

Master theorem

$T(n) = aT(n/b) + n^d$

$T(n) = 5T(n/7) + 3n^2$

$a = 5$ compare a to b^d

$b = 7$

$d = 2$ $\quad 5 < 7^2 = 49$

$$T(n) = \Theta(n^2)$$

Assignment 5

#2: Master Theorem

a. $T(n) = 5T\left(\frac{n}{9}\right) + n^7$

$a = 5, \quad b = 9, \quad d = 7$

$5 < 9^7$

$T(n) = \Theta(n^7)$

b. $T(n) = 3T\left(\frac{n}{3}\right) + 7n$

$a = 3, \quad b = 3, \quad d = 1$

$3 = 3^1$

$T(n) = \Theta(n \log n)$

c. $T(n) = 15T\left(\frac{n}{4}\right) + 100$

$= 15T(n/4) + 100n^0 \Rightarrow 100(1)$

$a = 15, \quad b = 4, \quad d = 0$

$15 > 4^0 = 1$

$T(n) = \Theta(n^{\log_4 15})$

d. $T(n) = 9T\left(\frac{n}{3}\right) + n^2$

$a = 9, \quad b = 3, \quad d = 2$

$9 = 3^2 = 9$

$T(n) = \Theta(n^2 \log n)$

e. $T(n) = 6T\left(\frac{n}{5}\right) + n^3$

$a = 6, \quad b = 5, \quad d = 3$

$6 < 5^3 = 15$

$T(n) = \Theta(n^3)$

Assignment 5		**#3: Radix Sort**							
BOY, RIS, GUA, ATL, SUC, CES, QUI, PUT, ANT, VAU, HUI, VAL, SAN, SAN, CAQ, VIC, CAL, LA_GUA, TOL, META, CAU, BOL, AMA, MAG, CHO, NSA, GUA, CAS, COR									
Pass 1									
_									
A	GUA	LA_GUA	META	AMA	NSA				
B									
C	SUC	VIC							
D									
E									
F									
G	MAG	GUA							
H									
I	QUI	HUI							
J									
K									
L	ATL	VAL	CAL	TOL	BOL				
M									
N	SAN	SAN							
O	CHO								
P									
Q	CAQ								
R	COR								
S	RIS	CES	CAS						
T	PUT	ANT							
U	VAU	CAU							
V									
W									
X									
Y	BOY								
Z									
GUA, LA_GUA, META, AMA, NSA, SUC, VIC, MAG, GUA, QUI, HUI, ATL, VAL, CAL, TOL, BOL, SAN, SAN, CHO, CAQ, COR, RIS, CES, CAS, PUT, ANT, VAU, CAU, BOY									

Assignment 5		**#3: Radix Sort**							
Pass 2									
_									
A	MAG	VAL	CAL	SAN	SAN	CAQ	CAS	VAU	CAU
B									
C									
D									
E	CES								
F									
G									
H	CHO								
I	VIC	RIS							
J									
K									
L									
M	AMA								
N	ANT								
O	TOL	BOL	COR	BOY					
P									
Q									
R									
S	NSA								
T	META	ATL							
U	GUA	LA_GUA	SUC	GUA	QUI	HUI	PUT		
V									
W									
X									
Y									
Z									
MAG, VAL, CAL, SAN, SAN, CAQ, CAS, VAU, CAU, CES, CHO, VIC, RIS, AMA, ANT, TOL, BOL, COR, BOY, NSA, META, ATL, GUA, LA_GUA, SUC, GUA, QUI, HUI, PUT									

Assignment 5		**#3: Radix Sort**							
Pass 3									
_									
A	AMA	ANT	ATL						
B	BOL	BOY							
C	CAL	CAQ	CAS	CAU	CES	CHO	COR		
D									
E									
F									
G	GUA	LA_GUA	GUA						
H	HUI								
I									
J									
K									
L									
M	MAG	META							
N	NSA								
O									
P	PUT								
Q	QUI								
R	RIS								
S	SAN	SAN	SUC						
T	TOL								
U									
V	VAL	VAU	VIC						
W									
X									
Y									
Z									
AMA, ANT, ATL, BOL, BOY, CAL, CAQ, CAS, CAU, CES, CHO, COR, GUA, LA_GUA, GUA, HUI, MAG, META, NSA, PUT, QUI, RIS, SAN, SAN, SUC, TOL, VAL, VAU, VIC									

Assignment 5		**#3: Radix Sort**							
Pass 4									
_	LA_GUA								
A	AMA	ANT	ATL						
B	BOL	BOY							
C	CAL	CAQ	CAS	CAU	CES	CHO	COR		
D									
E									
F									
G	GUA	GUA							
H	HUI								
I									
J									
K									
L									
M	MAG	META							
N	NSA								
O									
P	PUT								
Q	QUI								
R	RIS								
S	SAN	SAN	SUC						
T	TOL								
U									
V	VAL	VAU	VIC						
W									
X									
Y									
Z									
LA_GUA, AMA, ANT, ATL, BOL, BOY, CAL, CAQ, CAS, CAU, CES, CHO, COR, GUA, GUA, HUI, MAG, META, NSA, PUT, QUI, RIS, SAN, SAN, SUC, TOL, VAL, VAU, VIC									

Assignment 5		**#3: Radix Sort**							
Pass 5									
_									
A	AMA	ANT	ATL	LA_GUA					
B	BOL	BOY							
C	CAL	CAQ	CAS	CAU	CES	CHO	COR		
D									
E									
F									
G	GUA	GUA							
H	HUI								
I									
J									
K									
L									
M	MAG	META							
N	NSA								
O									
P	PUT								
Q	QUI								
R	RIS								
S	SAN	SAN	SUC						
T	TOL								
U									
V	VAL	VAU	VIC						
W									
X									
Y									
Z									
AMA, ANT, ATL, LA_GUA, BOL, BOY, CAL, CAQ, CAS, CAU, CES, CHO, COR, GUA, GUA, HUI, MAG, META, NSA, PUT, QUI, RIS, SAN, SAN, SUC, TOL, VAL, VAU, VIC									

Assignment 5		**#3: Radix Sort**							
Pass 6									
_									
A	AMA	ANT	ATL						
B	BOL	BOY							
C	CAL	CAQ	CAS	CAU	CES	CHO	COR		
D									
E									
F									
G	GUA	GUA							
H	HUI								
I									
J									
K									
L	LA_GUA								
M	MAG	META							
N	NSA								
O									
P	PUT								
Q	QUI								
R	RIS								
S	SAN	SAN	SUC						
T	TOL								
U									
V	VAL	VAU	VIC						
W									
X									
Y									
Z									
AMA, ANT, ATL, BOL, BOY, CAL, CAQ, CAS, CAU, CES, CHO, COR, GUA, GUA, HUI, LA_GUA, MAG, META, NSA, PUT, QUI, RIS, SAN, SAN, SUC, TOL, VAL, VAU, VIC									

Assignment 5		**#4: Double Hasing**			
62, 62, 69, 70, 4, 103, 85, 27, 86, 40, 96, 77, 23					
				Hash Table	
Key 62				0	
int h1 (int key) {				1	
int x = (|key – 9|) * (key + 13);	x = (|62– 9|) * (62 + 13);	x = 3975		2	
x = x / 15;	x = 3975 / 15	x = 265		3	
x = x + key;	x = 265 + 62	x = 327		4	
x = x % M;	x = 327 % 7	x = 5		5	62
return x;	x = 5			6	
Home Slot:	5				
Collisions:	0				
Inserted slot:	5				
Key 62					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|62– 9|) * (62 + 13);	x = 3975			
x = x / 15;	x = 3975 / 15	x = 265			
x = x + key;	x = 265 + 62	x = 327			
x = x % M;	x = 327 % 7	x = 5			
return x;	x = 5	COLLISION			
h2(key) = power(key)					
x = key^3 + 1	x = 62^3 + 1	x = 238329			
	x = 238329 % 7	x = 0			
Probe sequence:					
x = (hash(x) + i * h2(x)) % M	x = (5 + i * 0) % 7				
i = 0	x = (5 + 0 * 0) % 7	x = 5			
i = 1	x = (5 + 1 * 0) % 7	x = 5			
i = 2	x = (5 + 2 * 0) % 7	x = 5			
i = 3	x = (5 + 3 * 0) % 7	x = 5			
	x = (5 + i * 0) % 7	infinite loop			
Home Slot:	5				
Collisions:	4				
REHASH	2*M = 2 * 7 = 14	Next prime -> 17			

Key 62 (1st in list)					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|62– 9|) * (62 + 13);	x = 3975			
x = x / 15;	x = 3975 / 15	x = 265			
x = x + key;	x = 265 + 62	x = 327			
x = x % M;	x = 327 % 17	x = 4			
return x;	x = 4				
Home Slot:	4				
Collisions:	0				
Inserted slot:	4				
Key 62 (2nd in list)					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|62– 9|) * (62 + 13);	x = 3975			
x = x / 15;	x = 3975 / 15	x = 265			
x = x + key;	x = 265 + 62	x = 327			
x = x % M;	x = 327 % 17	x = 4			
return x;	x = 4	COLLISION			
h2(key) = power(key)					
x = key^3 + 1	x = 62^3 + 1	x = 238329			
	x = 238329 % 17	x = 6			
Probe sequence:					
x = (hash(x) + i * h2(x)) % M	x = (4 + i * 6) % 17				
i = 0	x = (4 + 0 * 6) % 17	x = 4	COLLISION		
i = 1	x = (4 + 1 * 6) % 17	x = 10			
Home Slot:	4				
Collisions:	2				
Inserted slot:	10				
Key 69					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|69– 9|) * (69 + 13);	x = 4920			
x = x / 15;	x = 4920 / 15	x = 328			
x = x + key;	x = 328 + 69	x = 397			
x = x % M;	x = 397 % 17	x = 6			
return x;	x = 6				
Home Slot:	6				
Collisions:	0				
Inserted slot:	6				

Key 70					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|70– 9|) * (70 + 13);	x = 5063			
x = x / 15;	x = 5063 / 15	x = 337			
x = x + key;	x = 337 + 70	x = 407			
x = x % M;	x = 407 % 17	x = 16			
return x;	x = 16				
Home Slot:	16				
Collisions:	0				
Inserted slot:	16				
Key 4					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|4– 9|) * (4 + 13);	x = 85			
x = x / 15;	x = 85 / 15	x = 5			
x = x + key;	x = 5 + 4	x = 9			
x = x % M;	x = 9 % 17	x = 9			
return x;	x = 9				
Home Slot:	9				
Collisions:	0				
Inserted slot:	9				
Key 103					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|103– 9|) * (103 + 13);	x = 10904			
x = x / 15;	x = 10904 / 15	x = 726			
x = x + key;	x = 726 + 103	x = 829			
x = x % M;	x = 829 % 17	x = 13			
return x;	x = 13				
Home Slot:	13				
Collisions:	0				
Inserted slot:	13				
Key 85					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|85– 9|) * 85 + 13);	x = 7448			
x = x / 15;	x = 7448 / 15	x = 496			
x = x + key;	x = 496 + 85	x = 581			
x = x % M;	x = 581 % 17	x = 3			
return x;	x = 3				
Home Slot:	3				
Collisions:	0				
Inserted slot:	3				

Key 27					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|27– 9|) * (27 + 13);	x = 720			
x = x / 15;	x = 720 / 15	x = 48			
x = x + key;	x = 48 + 27	x = 75			
x = x % M;	x = 75 % 17	x = 7			
return x;	x = 7				
Home Slot:	7				
Collisions:	0				
Inserted slot:	7				
Key 86					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|86– 9|) * (86 + 13);	x = 7623			
x = x / 15;	x = 7623 / 15	x = 508			
x = x + key;	x = 508 + 86	x = 594			
x = x % M;	x = 594 % 17	x = 16			
return x;	x = 16	COLLISION			
h2(key) = power(key)					
x = key^3 + 1	x = 86^3 + 1	x = 636057			
	x = 636057 % 17	x = 2			
Probe sequence:					
x = (hash(x) + i * h2(x)) % M	x = (16 + i * 2) % 17				
i = 0	x = (16 + 0 * 2) % 17	x = 16	COLLISION		
i = 1	x = (16 + 1 * 2) % 17	x = 1			
Home Slot:	16				
Collisions:	2				
Inserted slot:	1				
Key 40					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|40– 9|) * (40 + 13)	x = 1643			
x = x / 15;	x = 1643 / 15	x = 109			
x = x + key;	x = 109 + 40	x = 149			
x = x % M;	x = 149 % 17	x = 13			
return x;	x = 13	COLLSION			
h2(key) = power(key)					
x = key^3 + 1	x = 40^3 + 1	x = 64001			
	x = 64001 % 17	x = 13			
Probe sequence:					
x = (hash(x) + i * h2(x)) % M	x = (13 + i * 13) % 17				
i = 0	x = (13 + 0 * 13) % 17	x = 9	COLLISION		
i = 1	x = (13 + 1 * 13) % 17	x = 5			
Home Slot:	13				
Collisions:	2				
Inserted slot:	5				

Key 96					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|96– 9|) * (96 + 13);	X = 9483			
x = x / 15;	x = 9483 / 15	x = 632			
x = x + key;	x = 632 + 96	x = 728			
x = x % M;	x = 728 % 17	x = 14			
return x;	x = 14				
Home Slot:	14				
Collisions:	0				
Inserted slot:	14				
Key 77					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|77– 9|) * (77 + 13);	x = 6120			
x = x / 15;	x = 6120 / 15	x = 408			
x = x + key;	x = 408 + 77	x = 484			
x = x % M;	x = 484 % 17	x = 8			
return x;	x = 8				
Home Slot:	8				
Collisions:	0				
Inserted slot:	8				
Key 23					
int h1 (int key) {					
int x = (|key – 9|) * (key + 13);	x = (|23– 9|) * (23 + 13);	x = 504			
x = x / 15;	x = 504 / 15	x = 33			
x = x + key;	x = 33 + 23	x = 56			
x = x % M;	x = 56 % 17	x = 5			
return x;	x = 5	COLLISION			
h2(key) = power(key)					
x = key^3 + 1	x = 23^3 + 1	x = 12168			
	x = 12168 % 17	x = 13			
Probe sequence:					
x = (hash(x) + i * h2(x)) % M	x = (5 + i * 13) % 17				
i = 0	x = (5 + 0 * 13) % 17	x = 5	COLLISION		
i = 1	x = (5 + 1 * 13) % 17	x = 1	COLLISION		
i = 2	x = (5 + 2 * 13) % 17	x = 14	COLLISION		
i = 3	x = (5 + 3 * 13) % 17	x = 10			
Home Slot:	5				
Collisions:	4				
Inserted slot:	10				

Hash Table					
0					
1	86				
2					
3	85				
4	62				
5	40				
6	69				
7	27				
8	77				
9	4				
10	23				
11					
12					
13	103				
14	96				
15					
16	70				

Assignment 5

#7: Algorithm Analysis

5. Time complexity is O(n*m), where n is the number of strings and m is the length of the longest string, because we loop through each character position (m) from right to left (LSD). We process all strings, putting each into buckets and then into an array. Thus, time complexity is O(n * m).

Space Complexity is O(n + k), where n is the number of strings and k is the size of the character set (256 for ASCII), because each string is stored in a bucket and the bucket map requires space proportional to k. Since k is constant, space complexity is O(n).

6. Time complexity is $O(n^3 * m)$, where n is the number of words in the array and m is the maximum word length, because we check all possible triplets of the words and concatenate them with the nested loops, $O(n^3)$. We then check if the result is a palindrome, O(m). Thus, time complexity is $O(n^3 * m)$

Space complexity is $O(n^3 + n * m)$, where n is the number of words in the array and m is the maximum word length, because we store all possible palindromes in the result list, $O(n^3)$. Converting all the input words to lowercase creates O(n * m) space because each word can have a length up to m and we have to do it for each word in the array, n. Thus, space complexity is $O(n^3 + n * m)$.